

Trail Blazer 2: A Trained Linear SVM Obtained by Hill Climbing on Uniform Noise in Weight Space

Brian Mingus

May 2022

We demonstrate that a trained linear Support Vector Machine (SVM) can be obtained by adding uniform noise to the weights and checking to see if performance is improved on the entire dataset at each trial. We learn the same linear scoring function as used in the `SGDClassifier` in `scikit-learn`, for which this method also works, except that we omit the intercept.

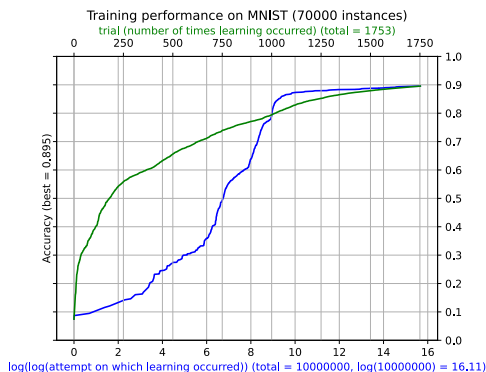
$$f(x) = W^T x$$

The PyTorch scoring function implements One-vs-All classification. The weights W start at 0 and uniform noise samples U are drawn on the interval $[0, 1)$.

$$f(X, y, W, U) := \text{sum}([\text{argmax}(X[W + U]) == y] > 1)$$

The method is sensitive to the solution learned during early learning and may require several restarts to match the performance of Stochastic Gradient Descent on normal noise in input space. A good initialization can be detected within the first 10,000 attempts, which is fast on the GPU. The errors are interdependent - 1 may look like 7 which may look like 9 and these mistakes entangle the hyperplanes for each class. With the present selection function unlearning may occur, but only if it improves performance overall. Not being able to search the entire space through unlearning, the model learns increasingly slowly over time as gradients become increasingly difficult to discover. Thus far we have not discovered the end of learning.

We demonstrate training performance on the full MNIST dataset. For each trial, uniform noise was added to the weights and inference was run on MNIST. If the number of samples that were correctly classified increased, the weight increase was kept.



1 Implementation

```
from math import log
import pickle
import time
import os

from torch import Tensor, argmax, zeros, rand, eq, sum

import torch
torch.set_printoptions(sci_mode=False, precision=30)
torch.set_default_tensor_type('torch.cuda.FloatTensor')

from matplotlib.ticker import MaxNLocator
from sklearn.datasets import fetch_openml
import matplotlib.pyplot as plt

mnist = fetch_openml('mnist_784', cache=True)
X = Tensor(mnist.data.values).to_sparse()
y = Tensor(mnist.target.astype('int64'))

n_trials = 10000

W = zeros(size=(784, 10))
U = rand(size=(784, 10))
results = Tensor(3000, 2).long()
attempts = Tensor([0]).long()
trial = Tensor([0]).long()
best = Tensor([0]).long()
correct = Tensor([0]).long()
_1 = Tensor([1]).long()

start = time.time()

for i in range(n_trials):

    correct = sum(eq(argmax(X @ (W + U.uniform_())), 1), y))

    if correct > best:

        W += U

        best = correct

        results[trial, 0] = attempts
        results[trial, 1] = best

        trial += _1

    attempts += _1

print(attempts, trial, best, best / 70000, time.time() - start)

with open("results.pickle", "wb") as f:
    pickle.dump(results, f)

with open("W.pickle", "wb") as f:
    pickle.dump(W, f)

_trials, _attempts, _accuracies = [], [], []

for t in range(trial):

    _attempt = results[t, 0].item()
    _accuracy = results[t, 1].item() / 70000

    if _attempt == 0:
        _attempts.append(0)
    else:
        _attempts.append(log(_attempt))

    _trials.append(t)

    _accuracies.append(_accuracy)

fig, ax1 = plt.subplots()
ax2 = ax1.twinx()

ax1.tick_params(labelright=True, labelleft=True)
ax1.yaxis.tick_right()
ax1.yaxis.set_ticks_position('both')

ax1.yaxis.set_major_locator(MaxNLocator(10))
ax1.xaxis.set_major_locator(MaxNLocator(10))

plt.title("Training performance on MNIST (70000 instances)")

plt.ylim(0, 1.0)

ax1.plot(_attempts, _accuracies, color="blue")
ax1.set_ylabel(f"Accuracy (best = {str(best.item()/70000)[:5]})")
```

```

ax1.set_xlabel(f"log(log(attempt on which learning occurred)) (total = {attempts.item()}, log({attempts.item()}) = {str(log(attempts.item()))[:5]})", color="blue")

ax2.plot(_trials, _accuracies, color="green")
ax2.set_xlabel(f"trial (number of times learning occurred) (total = {trial.item()})", color="green")

plt.grid(axis='both', which='both')
ax1.grid(axis='both', which='both')

fig.savefig("hc.svg")
plt.show()

```